

Documentazione Progetto Reti Informatiche

Anno Accademico 2025/2026, matricola pari

Davide Squeri

19/02/2025

Documentazione

Scelta del Protocollo di Trasporto

TCP per le comunicazioni Utente - Lavagna. La TCP handshake funziona da HELLO, o meglio il comando HELLO inizia la TCP handshake.

Scelta del tipo di concorrenza per la Lavagna

Ho optato per rendere la lavagna iterativa (non concorrente) nel servire le connessioni con gli utenti poiché:

1. l'approccio iterativo risparmia overhead
2. la sincronizzazione è più semplice
3. non serve concorrenza perché la lavagna esegue solo operazioni I/O bound leggere

Tengo dunque traccia degli utenti connessi tramite una lista, in modo da non dover definire un numero massimo di utenti contemporaneamente connessi. Con un solo thread devo dunque, controllare se ci sono nuove connessioni, gestire quelle esistenti e controllare l'input da terminale (stdin), aggiungo tutti questi descrittori al controllo della select e gestisco in ordine stdin, le nuove connessioni ed infine le connessioni già stabilite.

Utilizzando questo approccio non posso permettermi di avere comunicazioni utente - lavagna a più fasi, ogni scambio di informazioni deve essere botta-riposta, senza raccolta di input da terminale in mezzo, altrimenti la lavagna perderebbe tempo per colpa dell'utente ed ignorerebbe richieste da altri utenti connessi.

Questo porta allo svantaggio per esempio che l'utente sia informato della disponibilità dell'ID della card creata, solo dopo averla inviata. Sarebbe possibile scomporre (seguendo l'esempio della CREATE CARD) il processo di creazione ed invio di una nuova card in più comunicazione, separando l'invio dell'ID della card e l'invio della descrizione dell'attività relativa, però aumenterebbe il numero di send/recv.

Struttura dei pacchetti e modalità di scambio dei Dati

Per quanto riguarda le comunicazioni utente - lavagna, avendo optato per chiudere gli scambi di informazioni in al più un messaggio da parte dell'utente uno da parte della lavagna, ho la necessità di inviare più informazioni, anche di natura diversa in una stessa `send()`.

Ho deciso che i primi 4 byte di ogni messaggio sono l'ID del comando (`uint32_t`), in base a quello la lavagna (o l'utente) sa come interpretare il resto del messaggio. Per esempio la `CREATE_CARD` invia un messaggio costruito come segue:

```
4byte: ID_CREATE_CARD
4byte: ID_nuova_card
max1024byte: descrizione_attività
1byte: '\0'
```

Questi campi non sono separati o segnalati, la lavagna legge i primi 4 byte e riconosce il comando `CREATE_CARD`, da questo sa che i prossimi 4 byte sono l'ID della card e tutto

quello che segue (massimo 1024 byte) è la descrizione del messaggio, terminata da '0'.

I numeri sono portati in endianness di rete prima di essere inviati, e riportati in endianness di sistema all'arrivo.

Gestione dei socket

Scelte Varie

Ho optato per non rendere bloccante l'assegnazione di una Card: un utente quando riceve una card (Doing) non si blocca e continua a poter ricevere comandi da terminale.

Quando un utente riceve una Card, non si blocca, inoltre può ricevere nuove Card, per ogni Card che gli sono già state assegnate, il suo costo base sale, in modo da essere scelto più difficilmente.

Istruzioni sulla Compilazione

Comando per compilare il progetto:

```
make all
```

Comando per rimuovere i file oggetto:

```
make clean
```

Il comando crea i due file eseguibili richiesti nella root directory del progetto. Per eseguirli si può proseguire come indicato nelle specifiche del progetto.

1 ToDo

1. check ID utente
2. timeout per utenti
3. HELP
4. AVAILABLE_CARD
5. comunicazioni utente-utente